

Integración de Servicios Cloud GCP

Permisos, servicios, automatización con IA y acceso remoto en Linux

Semana 03 - Sesión 06

Rel Guzman



**Universidad
Tecnológica
del Perú**

- ▶ ¿Para qué sirve `systemctl`?
- ▶ ¿Qué diferencia existe entre `start` y `enable`?
- ▶ ¿Ya instalaron Warp y Antigravity CLI?

Logro de la sesión

Al finalizar, podrás administrar permisos de archivos, reconocer qué servicios se ejecutan o inician con Ubuntu, supervisarlos con apoyo de IA, relacionar una aplicación Node con MySQL y explicar cómo SSH permite administrar un equipo remoto.

- ▶ ¿Qué tanto saben sobre servidores creados con Node.js?
- ▶ La clase pasada accedimos a un servidor de Node.js desde la red local y lo probamos con `curl`. ¿Se acuerdan qué comando usamos?

¿Para qué te servirá lo aprendido?

Podrás ejecutar scripts con permisos controlados, identificar servicios del sistema, decidir cuáles deben iniciar con Ubuntu y comprobar el estado de una aplicación Node junto con MySQL.

También podrás pedir a la IA un monitor sencillo y seguro, y usar SSH como base para administrar máquinas virtuales y servidores cloud desde otra terminal.

Ayuda con IA en la terminal

Para qué sirve

VS Code es un editor gráfico. Permite abrir la carpeta del curso, editar los scripts y usar una terminal integrada sin salir del editor.

1. Instala el paquete .deb

```
$ dpkg --print-architecture  
arm64  
$ cd ~/Descargas  
$ sudo apt install ./code_*.deb
```

Descarga primero el paquete para Debian/Ubuntu de tu arquitectura desde code.visualstudio.com/download.

2. Abre la carpeta del curso

```
$ code ~/Escritorio/curso_cloud
```

También puedes abrirlo desde **Actividades** buscando Visual Studio Code.

Para qué sirve

Warp es una terminal gráfica que puede explicar comandos, sugerir pasos y ayudar a trabajar con proyectos. En Ubuntu puede seguir utilizando Bash.

1. Comprueba la arquitectura

```
$ dpkg --print-architecture  
arm64
```

La salida también puede ser amd64.
Descarga el paquete .deb correspondiente.

2. Instala y abre Warp

```
$ cd ~/Descargas  
$ sudo apt install ./warp-terminal_*.deb  
$ warp-terminal
```

Descarga primero el paquete para Debian/Ubuntu desde warp.dev/download.

Para qué sirve

agy abre un agente de IA en la terminal. Puede revisar archivos del proyecto, explicar código y proponer cambios sujetos a tu confirmación.

Instalación oficial en Ubuntu

```
$ sudo apt update
$ sudo apt install curl -y
$ curl -fsSL \
  https://antigravity.google/cli/install.sh | bash
$ source ~/.bashrc
```

Ejemplo en el laboratorio

```
$ cd
~/Escritorio/curso_cloud/sesion06
$ agy
```

Solicitud de ejemplo:

Explica los permisos de
saludo.py.

Primer inicio y ruta del comando

El primer inicio puede abrir el navegador para iniciar sesión. Si aparece agy: `command not found`, agrega `export PATH="$HOME/.local/bin:$PATH"` a `~/.bashrc` y ejecuta `source ~/.bashrc`.

Reto

Los comandos no se entregan: descúbrellos tú. Puedes apoyarte en Warp o en agy para encontrarlos, pero anota cuál usaste para cada dato.

Direcciones de origen y salida

VM_IP: la IPv4 activa de la VM.

ROUTER_IP: la puerta de enlace por la que sale el tráfico de la VM.

Internet y destino

PUBLIC_IP: la IP con la que Internet ve tu salida.

DEST_IP: una IPv4 de Google; usa Google como DEST_NAME.

En la Mac: HOST_IP

Averigua la IPv4 que la Mac tiene en la red Wi-Fi y regístrala como HOST_IP. Primero necesitas saber qué interfaz corresponde al Wi-Fi.

Modo de red

Confirma en la configuración del hipervisor:
Red = Bridged. Registra MODE=Bridged.

Evidencia obligatoria

Anota junto a cada valor el comando utilizado. El modo puente se sustenta con la configuración de la VM, no con un comando dentro de Ubuntu.

En la VM: protocolo y puerto

Captura el tráfico hacia DEST_IP mientras abres google.com en Firefox y detén la captura cuando cargue la página. A partir de lo capturado deduce y registra PROTOCOL y PORT.

Completa la ficha

```
VM_IP = [VM_IP]
HOST_IP = [HOST_IP]
ROUTER_IP = [ROUTER_IP]
PUBLIC_IP = IP pública
DEST_NAME = Google
DEST_IP = [DEST_IP]
PROTOCOL = [PROTOCOL]
PORT = [PORT]
MODE = Bridged
```

Prompt final

Crea un diagrama de secuencia del tráfico cuando abro google.com en Firefox desde una VM Ubuntu. Usa la ficha proporcionada. Muestra la resolución DNS, la salida desde VM_IP hacia ROUTER_IP, la traducción a PUBLIC_IP, la conexión PROTOCOL a DEST_IP:PORT y la respuesta. Indica junto a cada dato el comando usado para obtenerlo.

Precisión del modo puente

Incluye HOST_IP como contexto de la Mac, pero no como salto: en modo Bridged, la VM participa directamente en la red local.

Gestión de permisos

Idea central

Ubuntu protege cada archivo indicando qué acciones puede realizar el **propietario**, el **grupo** y el **resto de usuarios**.

¿Quién?

u: propietario.

g: grupo.

o: otros usuarios.

¿Qué puede hacer?

r: leer.

w: modificar.

x: ejecutar o atravesar un directorio.

Principio práctico

Concede solo el permiso necesario para la tarea. Dar más permisos no corrige la causa de un problema.

Archivo antes de hacerlo ejecutable

```
rgap@ubuntu:~/Desktop$ ls -l saludo.py
-rw-rw-r-- 1 rgap rgap 0 Aug 27 15:34 saludo.py
```

-	rw-	rw-	r--
Es un archivo regular. Una d indicaría directorio.	El propietario puede leer y modificar; todavía no puede ejecutar.	El grupo rgap puede leer y modificar el archivo.	Los demás usuarios solo pueden leerlo.

El script necesita una línea de intérprete y permiso x

Contenido de saludo.py

```
#!/usr/bin/env python3
usuario = input('¿Cómo te llamas? ')
print(f'Hola, {usuario}')
```

La primera línea decide quién lo interpreta

#!/usr/bin/env python3 indica que Ubuntu debe buscar Python 3 y usarlo para ejecutar el archivo.

Primer intento

```
$ ./saludo.py
bash: ./saludo.py: Permiso denegado
```

El archivo existe, pero el propietario aún no posee permiso de ejecución.

chmod u+x concede ejecución solo al propietario

Cambio y comprobación

```
$ chmod u+x saludo.py
$ ls -l saludo.py
-rwxrw-r-- 1 rgap rgap 157 Aug 27 15:41 saludo.py
$ ./saludo.py
¿Cómo te llamas? Ana
Hola, Ana. El script ya tiene permiso de ejecución.
```

Cómo se lee

u: propietario.

+: agrega un permiso.

x: permite ejecución directa.

Diferencia importante

python3 saludo.py puede funcionar sin x porque ejecutas Python y este lee el archivo.
./saludo.py sí demuestra el permiso de ejecución.

Cambios concretos

```
$ chmod u+x saludo.py  
$ chmod g-w datos.txt  
$ chmod o-r secreto.txt
```

Cada comando expresa quién cambia, si agrega o quita, y qué permiso afecta.

Evita `chmod 777`

Permitir lectura, escritura y ejecución a todos suele ocultar el problema y ampliar innecesariamente el acceso.

Forma numérica

$r=4$, $w=2$, $x=1$. La suma se calcula para propietario, grupo y otros.

```
$ chmod 750 tarea.sh  
rwx r-x ---
```

1. Entra en ~/Escritorio/cursos_cloud/sesion06/demo.
2. Ejecuta `ls -l saludo.py`.
3. Retira x con `chmod u-x saludo.py`.
4. Prueba `./saludo.py` y explica el error.
5. Agrega x con `chmod u+x saludo.py`.
6. Ejecuta nuevamente el script.

Comprobación: `ls -l` debe comenzar con `-rwx`.

Resultado esperado

```
$ chmod u-x saludo.py
$ ./saludo.py
bash: ./saludo.py: Permiso denegado
$ chmod u+x saludo.py
$ ./saludo.py
¿Cómo te llamas? Luis
Hola, Luis. El script ya tiene permiso de
ejecución.
```

Servicios al iniciar Ubuntu

Tres piezas relacionadas

Un **servicio** realiza una tarea en segundo plano. Un archivo `.service` describe cómo ejecutarlo. `systemd` aplica esa descripción y `systemctl` permite consultarla o controlarla.

Consultar no modifica

```
$ systemctl status mysql
$ systemctl is-active mysql
```

Controlar requiere privilegios

```
$ sudo systemctl start mysql
$ sudo systemctl enable mysql
```

Dos preguntas diferentes

`active` responde “¿está funcionando ahora?”. `enabled` responde “¿está configurado para iniciar con Ubuntu?”.

Comando solicitado para el laboratorio

```
$ sudo systemctl list-unit-files --type service --all
UNIT FILE STATE PRESET
mysql.service enabled enabled
ssh.service disabled enabled
systemd-journald.service static -
```

Qué muestra

Los archivos de unidad instalados y su estado de habilitación. No indica por sí solo cuáles están ejecutándose.

Sobre `sudo` y `--all`

La consulta normalmente funciona sin `sudo`; `--all` hace explícito que no se filtrará por estado.

enabled Tiene enlaces de arranque y normalmente se iniciará al alcanzar su objetivo.

disabled Existe, pero no quedó configurado para iniciar automáticamente.

static No se habilita directamente; otra unidad puede activarlo como dependencia.

masked Está bloqueado y no puede iniciarse hasta retirar la máscara.

PRESET Es otra columna: muestra la política recomendada por el paquete o la distribución.

No confundas estado de archivo con estado actual

Un servicio puede estar `active` y `disabled`, o estar `inactive` y `enabled`.

Servicios activos ahora

```
$ systemctl list-units  
--type=service --state=running
```

Consulta unidades cargadas que se encuentran funcionando.

Servicios habilitados al arranque

```
$ systemctl list-unit-files  
--type=service --state=enabled
```

Filtra archivos configurados para iniciar con Ubuntu.

Comprobación individual

```
$ systemctl is-active mysql  
active  
$ systemctl is-enabled mysql  
enabled
```


Solicitud dentro de ~/Escritorio/curso_cloud/sesion06/demo

Crea `monitor_servicios.py`. Debe consultar `systemctl is-active` e `is-enabled` para `mysql.service` y `mi-api.service`, mostrar fecha y hora, y no usar `sudo`, `shell=True`, reinicios ni cambios. Antes de escribirlo, explica el plan.

Revisa antes de ejecutar

```
$ sed -n '1,220p'  
monitor_servicios.py
```

Confirma que solo consulte y que use una lista de argumentos segura.

Límite explícito

El monitor informa. No debe reiniciar servicios automáticamente ni solicitar permisos de administrador.

Permiso y ejecución

```
$ chmod u+x monitor_servicios.py
$ ./monitor_servicios.py
Monitoreo: 2026-08-27 16:20:00
mysql.service activo=active inicio=enabled
mi-api.service activo=active inicio=enabled
```

Qué integra

1. Un script Python.
2. Permiso de ejecución.
3. Consultas a `systemctl`.
4. Revisión humana de la salida de IA.

Si un servicio no existe

La salida puede indicar `inactive`, `not-found` o un mensaje equivalente. Primero confirma el nombre con `list-unit-files`.

Servicios con Node y MySQL

Aplicación Node

Publica una API en el puerto 8000. Node.js no crea un servicio genérico `node.service`; nosotros definiremos `mi-api.service`.

Servidor MySQL

Mantiene disponible el motor de base de datos. En Ubuntu, el paquete `mysql-server` suele administrarse como `mysql.service`.

Preparación del laboratorio

```
$ sudo apt update
$ sudo apt install nodejs mysql-server -y
$ node --version
$ systemctl status mysql --no-pager
```

Comprueba el nombre real

Si usas MariaDB u otro paquete, el nombre puede cambiar. Búscalo antes con `systemctl list-unit-files '*mysql*' '*mariadb*'.`

Idea principal de app.js

```
const http = require('http');
const net = require('net');
net.createConnection(
  { host: '127.0.0.1', port: 3306 }
);
server.listen(8000, '127.0.0.1');
```

Respuesta de salud

Si MySQL acepta la conexión, la API devuelve estado 200. Si no responde, devuelve 503.

Alcance

Aquí verificamos disponibilidad del motor; las consultas SQL y credenciales se trabajarán por separado.

Archivo de unidad

```
[Unit]
Description=API Node de la Sesión 06
Requires=mysql.service
After=network.target mysql.service
[Service]
Type=simple
User=rgap
WorkingDirectory=/home/rgap/Escritorio/curso_cloud/sesion06/demo
ExecStart=/usr/bin/node app.js
Restart=on-failure
[Install]
WantedBy=multi-user.target
```

Cómo se interpreta

Requires: declara la dependencia.

After: ordena el inicio.

ExecStart: ejecuta Node.

WantedBy: permite habilitar el arranque.

Importante

Ajusta `User` y `WorkingDirectory` a tu cuenta. `After=mysql.service` ordena el inicio, pero no garantiza que MySQL ya acepte conexiones; por eso comprobamos el puerto.

Instala la unidad e inicia ambos servicios

```
$ sudo install -m 644 mi-api.service  
/etc/systemd/system/mi-api.service  
$ sudo systemctl daemon-reload  
$ sudo systemctl enable --now  
mysql.service mi-api.service
```

Comprueba

```
$ systemctl is-active  
mysql mi-api  
active  
active  
$ curl 127.0.0.1:8000  
{"node": ".activo",  
"mysql": ".alcanzable"}
```

Resultado: MySQL y la API quedan activos ahora y habilitados para el próximo inicio de Ubuntu.

Si la API no responde

```
$ systemctl status mi-api  
--no-pager  
$ journalctl -u mi-api -n 20  
--no-pager
```

Comprueba puertos y dependencia

```
$ ss -ltn | grep -E  
' :3306| :8000 '  
$ systemctl list-dependencies  
mi-api.service
```

Orden recomendado

Lee el estado y el registro antes de reiniciar. El mensaje de error suele indicar si falta el archivo, la ruta, el usuario o la dependencia.

Introducción a SSH

Qué es

SSH significa *Secure Shell*. Permite autenticarse en otro equipo y ejecutar comandos a través de una conexión protegida.

Dónde se usa

Máquinas virtuales, servidores Linux, instancias de Compute Engine y equipos de laboratorio.

Para qué se usa

Administrar servicios, revisar registros, copiar archivos y desplegar aplicaciones.

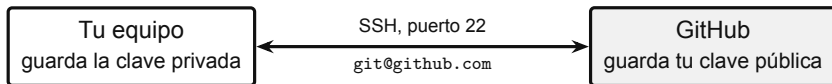
Qué necesita

Un cliente SSH, un servidor SSH accesible, usuario, dirección y un método de autenticación.

Modelo mental

Tu terminal es el cliente. La VM escucha normalmente en el puerto 22 y ejecuta allí los comandos autorizados.

GitHub permite trabajar por SSH y esto explica por qué



Te identifica

Registras tu clave pública en la cuenta. Tu clave privada demuestra quién eres sin enviar contraseña.

Protege el envío

El código y las credenciales viajan por un canal cifrado, aunque uses una red compartida.

Se nota en la URL

`https://github.com/...` usa la web; `git@github.com:...` usa SSH.

La misma idea que en una VM

GitHub, tu máquina virtual y un servidor cloud aceptan SSH por lo mismo: identifica al usuario y cifra la conexión. Si pierdes el equipo, se revoca esa clave desde la cuenta sin cambiar nada más.

Por qué importan

Una misma dirección IP atiende varios servicios a la vez. El puerto indica a cuál de ellos llega cada conexión.

Acceso y web

22 SSH: administración remota.

80 HTTP: web sin cifrar.

443 HTTPS: web cifrada.

53 DNS: resolución de nombres.

Datos y aplicaciones

3306 MySQL.

5432 PostgreSQL.

8000 y 3000: aplicaciones Node en desarrollo.

25 y 587 SMTP: correo saliente.

Ya los usaste en esta sesión

443 al abrir Google en Firefox, 3306 cuando `mi-api` comprueba MySQL y 8000 al consultar la API.
`ss -ltn` muestra qué puertos escucha tu equipo.



gracias



Universidad
Tecnológica
del Perú

- ▶ GNU Coreutils: manual de `chmod`; páginas de manual de `ls`, `chmod` y `env`.
- ▶ `systemd`: páginas de manual de `systemctl`, `systemd.unit` y `systemd.service`, freedesktop.org/software/systemd/man.
- ▶ Documentación de Ubuntu Server: OpenSSH y MySQL, documentation.ubuntu.com/server.
- ▶ Documentación de Node.js: módulos nativos `http` y `net`, nodejs.org/api.
- ▶ Documentación oficial de Warp para Linux, docs.warp.dev; documentación oficial de Google Antigravity CLI, antigravity.google/docs/cli.
- ▶ Documentación oficial de Visual Studio Code para Linux, code.visualstudio.com/docs/setup/linux.
- ▶ Material base proporcionado: *Fundamentos de Linux — Programa completo*; identidad visual y estructura docente de la Sesión 05.